SecChaos-RL: A Threat-Aware Self-Adaptive Resilience Engineering Framework for Secure Software Systems

Thurbilli Karthik[1*], Michael Arock[1] and Vinay Raj[2]

[1]Department of Computer Applications, National Institute of Technology, Tiruchirappalli, 620015, Tamil Nadu, India.
[2]Department of Computational & Data Science, National Institute of Technology, Warangal, 506004, Telangana, India.

*Corresponding author(s). E-mail(s): 405124053@nitt.edu;
Contributing authors: michael@nitt.edu; vinayraj@nitw.ac.in;

Abstract

Modern software systems are being deployed more and more as a distributed microservice architecture, increasing the attack surface and increasing the difficulty to be resilient to security threats and operational failures. Security Chaos Engineering (SCE) enhances the security resiliency by introducing intentional security issues, but the current practices of SCE are mostly static and manual. In this paper, we present a reinforcement learning (RL) framework for autonomous threat-aware mitigation in SCE environments (SecChaos-RL). The security mitigation process is represented as a Markov Decision Process (MDP), with which the agent learns adaptive mitigation policies based on an ongoing interaction with dynamically injected security faults. The proposed framework is compared with two state of the art algorithms, namely Deep Q-Network (DQN) and Advantage Actor-Critic (A2C), as well as Proximal Policy Optimization (PPO), in a simulated SCE environment with 30 independent random seeds. SecChaos-RL scores 88.95% secure, which would take up 4.91% of the simulated attack, service availability is maintained at 96.26%, and the Mean Time to Recovery (MTTR) is reduced by 55.3% from PPO. The findings suggest that adaptive reinforcement learning is able to achieve the balance between security, availability, and recovery, and shows its potential of autonomous mitigations to enhance software resilience in the security chaos scenario.

Keywords: Security Chaos Engineering, Software Resilience, Reinforcement Learning, Software Security, AI for Software Engineering, Adaptive Security

1 Introduction

Resilient digital ecosystems are crucial in critical infrastructure, healthcare, finance, and cloud computing, where software systems are growing quickly and fast [1]. Distributed systems in the modern era are made up of hundreds of microservices, third party dependencies, and complex inter-service interactions that can be exploited by adversaries continuously [2]. Tools like static analysis, penetration testing, and vulnerability scanning are useful but do not provide a comprehensive understanding of system dynamics during an active security threat [3]. The study of improving the detection of software vulnerabilities in recent years has focused on the learning of features by machine learning and representations at program levels [4].

DevOps has revolutionized the way software is developed and delivered, with a focus on collaboration, automation, and continuous integration and deployment [2]. However, the speed and agility of DevOps practices could be affecting more traditional security controls and resulting in tension between speed and security assurance. It resulted in the creation of DevSecOps, the integration of security with DevOps. The integration is to make security part of the entire Software Development Life Cycle (SDLC) [2]. Security assessment for continuous deployment is still a big challenge in DevSecOps pipelines. Traditional security testing methods, such as vulnerability scanning, static application security testing (SAST) and dynamic application security testing (DAST) can provide information from a pre-production environment, but cannot reflect the actual behaviour of systems during security attacks in production [5]. Researchers have recently explored the integration of code static analysis with reasoning techniques for the detection of vulnerabilities like buffer overflows, reflecting ongoing advancements in boosting the effectiveness of static vulnerability analysis [6]. In addition, these techniques usually test components separately and not security properties resulting from the interactions of these components in a complex distributed system. Chaos Engineering has become an engineering method to look for systemic weaknesses, and it actively introduces faults in production-like environments [1]. Netflix introduced the approach with a Chaos Monkey tool and it has grown over the years into a good practice and being adopted by many tech companies. The idea behind Chaos Engineering is to create confidence in a system's resilience to turbulent conditions through a series of controlled experiments designed to cause failures in the system and see what happens [1]. A multi-vocal literature review presented in [7] analyzed 96 sources from both academic and grey literature to establish a comprehensive understanding of chaos engineering, including its definition, core functionalities, architectural components, and adoption drivers. The results of the work were a taxonomy of chaos engineering platforms, which included aspects like execution environments, automation modes, automation strategies, and deployment types. This taxonomy was developed to facilitate a comparison of the chaos engineering tools most relevant

to each other to assist organizations to match the right tool to their architectures or resilience requirements.

The typical Chaos Engineering deals with failures of availability and performance, while network partitions, instance terminations and resource exhaustion are more of a security vulnerability and need special treatment [5]. This divide has resulted in the creation of a Chaos Engineering extension, Security Chaos Engineering (SCE). Security Chaos Engineering is a paradigm shift for any organization to test and validate their security posture [3]. SCE is dynamic, unlike static evaluations or penetration tests that are performed periodically. By introducing security vulnerabilities like expired certificates, insecure access permissions, and vulnerability of dependencies, SCE evaluates how well the system can withstand, detect, and recover from security risks [5]. The present research stresses the fact that traditional cyber security models are fundamentally flawed, dealing with prevention of failures in intrinsically complicated systems. Rather, it is recommended to use security chaos engineering to test the reaction of systems to bad conditions [3]. Security fault support has become a part of the industrial initiatives' efforts to integrate security into the chaos engineering tooling. Some major platforms have incorporated a limited security fault injection features, like ChaosBlade and LitmusChaos [8, 9]. Yet, these methods are still largely manual, involving pre-prepared experiment scripts and post experiment analysis by humans.

The current SCE approaches have one major drawback: Reactive nature; a fault is injected and observations are recorded, and remediation requires a human intervention [7]. With attacks getting more sophisticated and systems even more complex, it is crucial to have autonomous remediation to ensure the real-time security posture. This constraint drives the need for embedding AI, specifically Reinforcement Learning (RL), into the design of self-adapting security systems that learn the best mitigation strategies through interaction with their environment. Reinforcement Learning has been shown to achieve tremendous performance across a wide range of complex decision-making tasks, such as playing games and controlling robots [10, 11]. The first work in this field laid the theoretical groundwork for RL as a way to learn through trial and error in interaction with an environment [12]. These have been further expanded by deep reinforcement learning, which allows the learning to be directly from the high dimensional sensory inputs [13]. With the latest developments in AI, software quality and security analytics have been impacted more and more. A machine-learning based approach to vulnerability detection has been examined feature-learning for learning security-relevant program representations [4]. Code static analysis has also been combined with reasoning-based approaches to improve the detection of vulnerabilities such as buffer overflows [6]. Beyond vulnerability detection, large language models have been applied to software quality tasks such as bug report classification [14] and automated test plan generation [15]. However, the increasing reliance on machine-learning-based software analytics also introduces new concerns, including susceptibility to adversarial attacks and the need for robust and explainable AI-driven decision making [16]. These developments demonstrate the growing role of AI in software quality and security, but existing

approaches primarily focus on detection, classification, analysis, or test generation rather than autonomous runtime adaptation and remediation.

RL has been applied in several domains of cybersecurity. Research has presented surveys on deep reinforcement learning methods for cyber security, including DRL-based security methods for cyber-physical systems, autonomous intrusion detection techniques, and multiagent DRL-based game theory simulations for defense strategies against cyberattacks [17]. Recent work has also reviewed the state-of-the-art autonomous penetration testing based on reinforcement learning, identified the main challenges and classified the existing literature into attack path planning and autonomous penetration testing frameworks [18]. Notable developments have introduced simulation environments for autonomous cyber agents, featuring simulation and emulation capabilities with a common interface to facilitate the rapid training of autonomous agents [19]. Their work demonstrates the feasibility of applying RL to autonomous cyber operations. Investigations into IoT security techniques based on machine learning, including supervised learning, unsupervised learning, and reinforcement learning, have addressed authentication, access control, secure offloading, and malware detection schemes [20]. Recent algorithmic advances have proposed improved reinforcement learning approaches that give different experiences different priorities to update policy based on reward, demonstrating improved training effectiveness [21].

However, the use of RL in Security Chaos Engineering has not been studied. Previous works have used RL to automate incident response in Security Operations Centers, but their approach assumes static network topology and attack paths, and does not model the dynamic fault injection, cascading microservice dependencies, and operational trade-offs that characterize continuous cloud-native environments [19]. There is a large research gap in combining chaos-driven vulnerability discovery and RL-driven autonomous remediation.

In this paper, we propose SecChaos-RL, a novel framework for Security Chaos Engineering with reinforcement learning to develop self-healing, resilient software systems. We tackle the central problem of autonomous security adaptation by modeling the problem as a multi-objective Markov Decision Process (MDP) where the agent learns to select the best mitigation actions in response to injected security faults.

The main contributions of this study are summarized as follows:

- A comprehensive Security Chaos Engineering environment is constructed to model eight representative categories of security faults, with realistic fault-state transitions and attack-success dynamics.
- We propose an adaptive mitigation framework based on reinforcement learning to autonomously select response actions under changing threat conditions.
- We propose a multi-dimensional security resilience evaluation model to quantify system robustness by combining assessments of threat severity, service availability and recovery effectiveness.
- An open and reproducible research framework is provided through a complete implementation built on industry-standard software libraries.

The paper is organized as follows. Section II reviews the related literature on Chaos Engineering, Security Chaos Engineering, and Reinforcement Learning, highlighting existing approaches and identifying research gaps addressed in this study. Section III presents the SecChaos-RL framework architecture and the details of the Security Chaos Environment. Section IV presents expected results and discussion. Section V addresses threats to validity. Section VI concludes with future work directions.

2 Related Work

This section reviews existing research across four domains: Chaos Engineering, Security Chaos Engineering, Reinforcement Learning for cybersecurity, and Reinforcement Learning for resilience management.

Traditional Chaos Engineering research has been mainly about infrastructure resilience. Basiri et al. [1] did a big study on chaos engineering principles and practices at Netflix, including identifying common fault types, underlying principles and challenges faced by organizations in conducting experiments in production environments. The work lead to the creation of chaos engineering as a formal approach to testing for reliability in a DevOps setting. Rosenthal and Jones [22] composed a comprehensive overview of chaos engineering principles, and the need to begin small, minimise blast radius, and automate experiments. Their work included real-world insights from practitioners from various technology companies, such as Google, Microsoft, Slack, and LinkedIn. The multivocal literature review presented in [7] brings together evidence found in 96 academic and grey literature resources, published in the period between January 2016 and April 2024, to provide a holistic picture of chaos engineering, covering its definition, key features, architectural elements, adoption motivations, and ongoing research. They suggested a single definition of chaos engineering, distilled out the key functionalities, components and drivers for taking it to scale, and proposed a taxonomy of chaos engineering platforms. The taxonomy features aspects such as execution environments, automation modes, automation strategies and deployment types, enabling organizations to customize their tool selection to their system architectures and resilience goals.

Such tools currently being used to record the industry adoption are Chaos Monkey [1], Gremlin [23] and Chaos Mesh [24] or AWS Fault Injection Simulator [25]. These are focused on infrastructure type failures (instance failures, network latency and disk saturation) as opposed to security vulnerabilities. Esen et al. [26] performed a systematic literature review of the chaos experiments performed with microservice architectures, and investigated 31 research papers and 38 tools that are essential for chaos injection methods like Chaos Toolkit, Gremlin and Chaos Mesh. They explored the role of centralized management of chaos in enabling coordination in complex distributed systems. Security Chaos Engineering is an explicit application of chaos principles to an attack vector and security control [5]. Unlike availability-centric chaos, SCE does not insert faults that mimic failures like a crashing application, but rather, it adds faults to simulate security breaches like

compromised credentials, misconfigured firewalls, expired certificates, or vulnerable software components. The emphasis moves from verifying the availability of the system to the detection, resistance and recovery to security threats.

A multivocal literature review conducted by Myrbakken and Colomo-Palacios [2] identified the security issues in DevSecOps pipelines, which showed that it is quite difficult to implement security in pipelines that can match DevOps agility. They found that traditional security approaches have not been able to keep up with the pace and flexibility of DevOps, but may offer significant advantages if it is done successfully. They followed up with their own work [5] which introduced Security Chaos Engineering (SCE) and proposed a solid approach for injecting security faults to test defense mechanisms, and showed how to integrate with CI/CD pipelines. This work is the first to propose a continuous security assessment methodology for DevOps environments, called SCE.

In [3] the authors feature an in-depth review of Security Chaos Engineering (SCE). They introduce principles and systematic strategies for introducing security faults, and approaches for assessing the resilience of a system, as well as the use of security experimentation in contemporary software development. They contend that the problem with cybersecurity is that there is no one defending against attacks, while there are pressures on engineering teams to design, build and operate secure systems. Their work highlights the fact that failure can't be avoided, there are shortcomings in mental models of systems, and the digital world is constantly changing. They coach on building contemporary security programs, and know how to make informed choices at every level of software delivery to foster resilience, comprehend complicated systems dynamics, balance technical and organizational choices, and use chaos experimentation to validate key assumptions about software quality and security.

Bedoya et al. [27] introduced the use of Large Language Models (LLM) to support the automation of threat discovery in the design phase, and the usage of Security Chaos Engineering to support the detection of security vulnerabilities that may not be detected by security tools. Through their case study, they illustrated the feasibility of their proposal in a retail company that requires secure software development within a short time. Some limited security fault support has been added to industrial tools like ChaosBlade [8] and LitmusChaos [9]. Current SCE implementations, however, come with many drawbacks: very few fault types, the manual design of experiments, and the lack of intelligent response mechanisms [3]. Automation and intelligent decision making in SCE has been recognized as a key research area.

Reinforcement Learning (RL) has proven to be a promising method for tackling intricate cybersecurity challenges, particularly in the field of decision-making. This demonstrated the potential of RL in complex decision-making with large state spaces, as Silver et al. [10] showed with their superhuman performance in the game of Go in combination with a deep neural network and tree search. Recent deep RL methods, such as value-based approaches like Deep Q-Network (DQN) [13] and policy gradient methods like Proximal Policy Optimization (PPO) [11], have made it possible to apply deep RL to high-dimensional, complex environments. Asynchronous methods for deep reinforcement learning were introduced by Mnih et al. [28] who presented asynchronous

versions of four basic RL algorithms which outperformed state-of-the-art algorithms on the Atari domain.

RL has been investigated in different areas for cybersecurity applications. Nguyen and Reddi [17] surveyed the various methods of DRL in cyber security and their applications to security methods in cyber-physical systems, autonomous intrusion detection and multi-agent DRL-based game theory simulations of defense strategies against cyberattacks. They build an extensive survey of the state-of-the-art about how DRL has been applied to security issues. Liu et al. [18] reviewed the state-of-the-art autonomous penetration testing based on reinforcement learning, identified the main challenges, and classified the existing literature into attack path planning and autonomous penetration testing frameworks. In their systematic review, they pointed out the possibilities and drawbacks of RL based automated security testing techniques. Standen et al. [19] proposed CybORG, a Gym environment for the development of autonomous cyber agents that provides both simulation and emulation capabilities through a unified interface, enabling efficient training and evaluation of reinforcement learning agents. The study demonstrated the effectiveness of reinforcement learning in autonomous cyber operations, particularly for incident response automation and adaptive decision-making in security operations. To address the security issues with IoT, Xiao et al. in [20] studied the machine learning-based IoT security techniques for authentication, access control, secure offloading, and malware detection schemes. They have shown the utility of RL to various security problems in the IoT domain.

Algorithmic contributions by Zheng et al. [21] propose a Reward-based Prioritization Proximal Policy Optimization (RP-PPO) algorithm that assigns different priorities to different experiences for policy update based on reward, demonstrating better training effectiveness compared to standard PPO methods. Gymnasium [29] offers a common interface for RL environments, facilitating interoperability of RL implementations over different environments and training algorithms. The stable-baselines3 library [30] offers a suite of well-documented and stable implementations of deep RL algorithms, with consistent interfaces. There is growing research at the RL / system resilience intersection. In the context of Security Operations Centers, Standen et al. [19] proposed an autonomous system for security operations through the automation of incident response with RL. They, however, only support fixed network topology representations and fixed attack paths, and lack a representation of dynamic fault injection, cascading microservice dependencies, and operational trade-offs of continuous cloud-native environments. Resilience research is embedded in the design of self-adaptive software systems.Self-adaptive software systems provide a wider framework for resilience studies. Salehie and Tahvildari [31] gave a thorough survey on self-adaptive software, proposing self-adaptation concerns (how, what, when and where) to categorize it. They point out the importance of closed loop systems with feedback loops that will adapt to changes in operation.

Garlan et al. [32] proposed a framework for self adaptation based on the architecture with reusable infrastructure called Rainbow. Rainbow addresses the challenges of handling a wide variety of systems and reducing costs in adding

external control to systems. Their "model" offers reusable infrastructure, as well as a way to specialize that infrastructure to specific systems. The authors of the Systems Correctness in the wild paper in Amazon Web Services, [33], surveyed the methods used in the field for systems correctness at AWS. Their work demonstrates the evolution of AWS from unit testing at build-time to a combination of formal and semi-formal testing techniques, such as property-based testing, deterministic simulation and fault injection as a service. Their experience with TLA+ and the P programming language illustrates the importance of rigorous verification approaches, but these are very different from the learning-based one we are proposing here.

Current approaches that focus on security fault injection and lack intelligent response, and those that use RL for isolated security tasks without chaos-driven validation. While previous work has explored the use of RL for automating incident response in SOsCs, these approaches are limited by static network topologies and attack paths, as well as a lack of consideration for the dynamic fault injection, cascading microservice dependencies, and operational trade-offs that characterize continuous cloud-native environments [19]. No literature has been found that discusses the use of RL in Security Chaos Engineering. The integration of the chaos-driven vulnerability discovery and RL-driven autonomous remediation remains under well-researched. Some industrial tools have been developed for adding limited security fault injection capabilities [8, 9] but these are still manual and don't use intelligent decisions making. Likewise, in the field of cyber security, RL has been used in different applications including [17, 18] but with each application focused in a specific security domain, without taking into account system-wide resilience or chaos-driven validation.

To close this loop and continuously improve security resilience, SecChaos-RL combines a comprehensive Security Chaos Engineering environment and security resilience driven adaptive mitigation via RL. Our framework overcomes the limitations of existing approaches by: (1) modeling different security fault types with real-world dynamics; (2) enabling autonomous selection of mitigation actions through reinforcement learning; (3) integrating multi-objective optimization considering security risk, service availability and resource efficiency; and (4) providing a reproducible research framework for comparative assessment and future extensions.

3 Methodology

The proposed SecChaos-RL framework is introduced, encompassing the overall framework, essential elements, and the operational workflow.

3.1 Architecture Overview

The overall architecture of SecChaos-RL is a closed-loop security resilience framework in which behaviours of the system are continuously monitored, controlled security faults are injected, security resilience outcomes are assessed and mitigation strategies are dynamically adjusted using reinforcement learning algorithms. It consists of six

interconnected aspects, all of which allow for independent experiments with security chaos and adaptive optimisation of security resilience.

1. **Security Fault Injector:** This is the component which is responsible for choosing and adding security faults to the target system. It continues to hold the taxonomy of faults and provides fault injection mechanisms to emulate real world security incidents. The injector works under the control of the RL agent who decides which fault occurs when and what fault it is, and uses exploration strategy to perform its task.
2. **Software Security Environment:** The software system that is the target of the security analysis, represented as a stateful entity having security attributes, service dependencies and behavioural patterns. It replicates system's responses to the faults injected like attack success probability, availability degradation, and recovery dynamics.
3. **Monitoring Layer:** It continuously observes the state of the environment and extracts metrics relevant to the security posture, system availability and operational health. The monitoring layer converts raw system data into a structured representation of the state required by the RL agent.
4. **RL Security Agent:** The core intelligence of the framework, this component implements the reinforcement learning algorithm. It receives state observations, selects mitigation actions according to its learned policy, and updates its parameters based on received rewards. The agent balances exploration of new strategies with exploitation of known effective responses.
5. **Mitigation Engine:** Upon receiving action commands from the RL agent, the mitigation engine translates abstract actions into concrete system operations such as certificate rotation, dependency patching, access control modification and executes them within the environment.
6. **Reward Evaluator:** This component computes the reward signal using the multi-objective reward function. It measures the security improvement, the impact on availability, the effectiveness of recovery and the operational costs, providing the feedback required for policy learning.

The Security Chaos Workflow is performed in discrete time steps as follows: (1) the environment is initialized with a baseline security configuration, (2) a security fault is injected according to the exploration policy, (3) the monitoring layer captures the resulting state, (4) the RL agent selects a mitigation action, (5) the mitigation engine executes the action, (6) the reward evaluator computes feedback, (7) the agent updates its policy. This cycle is repeated for the training episodes until convergence.

3.2 Security Chaos Environment

The Security Chaos Environment models a realistic software system subject to security-relevant faults. To systematically generate security faults and operational telemetry for training, we build upon systematic fault and data generation models for software testing [34]. This section defines the fault taxonomy and formal fault model.

Table 1 presents the eight fault categories modeled in SecChaos-RL, covering authentication, authorization, cryptographic, dependency, and configuration vulnerabilities.

Table 1 Security Fault Categories

ID	Fault Type	Description
F_1	Expired TLS Cert.	TLS cert. expiration leading to MITM susceptibility
F_2	Vuln. Dependency	Third-party library with known CVE
F_3	Secret Leakage	Exposed credentials in logs or env. variables
F_4	RBAC Misconfig.	Incorrect role-permission mapping
F_5	Weak Authentication	Disabled MFA or weak password policy
F_6	Excessive Privileges	Service account with unnecessary perms.
F_7	Missing Patch	Unpatched system or application vuln.
F_8	API Exposure	Unprotected internal API endpoint

We define a security fault as a tuple:

$$f_i = (type_i, severity_i, duration_i, scope_i, detectability_i) \tag{1}$$

where $type_i \in \{F_1, \ldots, F_8\}$ is the fault category, $severity_i \in [0, 1]$ represents the potential impact, $duration_i \in \mathbb{N}^+$ defines the fault persistence in time steps, $scope_i \in \{\text{local}, \text{service}, \text{system}\}$ indicates the affected components, and $detectability_i \in [0, 1]$ is the detection probability by existing monitoring.

The fault set for an episode is defined as:

$$\mathcal{F}_{episode} = \{f_1, f_2, \ldots, f_n\}, \quad n \sim \text{Poisson}(\lambda_f) \tag{2}$$

where λ_f controls the average number of faults per episode.

Fault injection triggers a state transition affecting multiple security dimensions simultaneously.

To capture the architecture of modern distributed microservices, we model service dependencies as a directed acyclic graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of services (e.g., Gateway, Authentication, User, Payment, Order, Inventory, Notification) and $\mathcal{E}$ represents directed dependencies. Each service $v \in \mathcal{V}$ maintains local risk $r(v) \in [0, 1]$ and availability $a(v) \in [0, 1]$ metrics.

When a security fault is introduced at service $u \in \mathcal{V}$, its risk is updated to $r(u) \leftarrow \min(1.0, r(u) + severity_i)$ and its availability is updated to $a(u) \leftarrow \max(0.0, a(u) - 0.1 \cdot severity_i)$. If the risk is greater than a threshold ($r(u) > 0.4$), cascading propagation to dependent services is triggered. The propagation to any dependent child service v

(where $(u, v) \in \mathcal{E}$) is given by:

$$r(v) \leftarrow \min\left(1.0, r(v) + 0.25 \cdot r(u) \cdot \gamma_{cascade}\right) \tag{3}$$

$$a(v) \leftarrow \max\left(0.0, a(v) - 0.15 \cdot [0.25 \cdot r(u) \cdot \gamma_{cascade}]\right) \tag{4}$$

where $\gamma_{cascade} \in [0, 1]$ is the cascade coupling factor (we use 0.6 in our environment). The aggregate risk and availability metrics are averaged over all the nodes in $\mathcal{G}$.

To stabilize the reinforcement learning for highly complex settings, we design a multi-stage curriculum scheduler $\mathcal{C}(e)$ mapped to the current training episode $e \in \{1, \ldots, 250\}$:

$$\mathcal{C}(e) = \begin{cases} 0 & \text{if } e < 50 \quad \text{(Threat Level 1: Baseline)} \\ 1 & \text{if } 50 \leq e < 100 \quad \text{(Threat Level 2: Moderate)} \\ 2 & \text{if } 100 \leq e < 150 \quad \text{(Threat Level 3: Severe)} \\ 3 & \text{if } e \geq 150 \quad \text{(Threat Level 4: Critical Chaos)} \end{cases} \tag{5}$$

Each level scales the environment parameters to increase security chaos: (1) **Threat Level 1** (Episodes 0–50) is a baseline environment where the fault arrival rate is low ($\lambda_f = 0.15$), faults are restricted to simple configuration issues (F_1, F_5), and adversarial attacks and cascading effects are inactive; (2) **Threat Level 2** (Episodes 50–100) is a moderate environment with an arrival rate of $\lambda_f = 0.25$ introducing faults $\{F_1, F_3, F_4, F_5\}$ and active attack simulation with a maximum of one concurrent attack ($A_{max} = 1$); (3) **Threat Level 3** (Episodes 100–150) represents a severe environment with an arrival rate of $\lambda_f = 0.35$ across faults $\{F_1, \ldots, F_6\}$ and concurrent attacks up to $A_{max} = 2$; and (4) **Threat Level 4** (Episodes 150–250) represents Critical Chaos, where full system complexity is enabled with arrival rate $\lambda_f = 0.45$, all fault types $(F_1, \ldots, F_8)$, concurrent attacks up to $A_{max} = 4$, and active dependency-graph cascade propagation.

3.3 Mathematical Formalization

This section provides the complete mathematical formulation of the SecChaos-RL problem as a Markov Decision Process.

3.3.1 Markov Decision Process Definition

The Security Chaos Environment is formalized as an MDP:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma) \tag{6}$$

3.3.2 State Space and Observation Vector

The underlying state space $\mathcal{S} \subset \mathbb{R}^9$ of the environment is defined by a nine-dimensional vector tracking core security risks and operational states, following

standard formulations for state-action representations in reinforcement learning [10, 13] and autonomous cyber operations [19]:

$$s_t = \begin{bmatrix} s_t^{cert} \\ s_t^{dep} \\ s_t^{sec} \\ s_t^{rbac} \\ s_t^{auth} \\ s_t^{patch} \\ s_t^{atk} \\ s_t^{avail} \\ s_t^{rec} \end{bmatrix} = \begin{bmatrix} \text{certificate_risk} \\ \text{dependency_risk} \\ \text{secret_risk} \\ \text{rbac_risk} \\ \text{authentication_risk} \\ \text{patch_level} \\ \text{attack_probability} \\ \text{availability} \\ \text{recovery_time} \end{bmatrix} \tag{7}$$

$s_t^{cert}, s_t^{dep}, s_t^{sec}, s_t^{rbac}, s_t^{auth} \in [0,1]$ are normalized risk scores, $s_t^{patch} \in [0,1]$ is a patch currency score, $s_t^{atk} \in [0,1]$ is the estimated attack success probability, $s_t^{avail} \in [0,1]$ is the system availability, and $s_t^{rec} \in [0, T_{max}]$ is the current recovery time metric.

However, to account for partial observability and supply the learning agent with diagnostic and historical context, the agent receives an augmented observation vector $o_t \in \mathbb{R}^{16}$:

$$o_t = [o_t^{(1)}, \ldots, o_t^{(16)}]^T \tag{8}$$

where the 16 elements correspond to: (1) $o_t^{(1:5)} = [s_t^{cert}, s_t^{dep}, s_t^{sec}, s_t^{rbac}, s_t^{auth}]^T$ (core security risks); (2) $o_t^{(6)} = \mathcal{L}_{threat}(s_t)$ (estimated aggregate threat severity); (3) $o_t^{(7)}$ is the vulnerability score (aggregate vulnerability index); (4) $o_t^{(8)}$ is the active attacks ratio (normalized active attacks count); (5) $o_t^{(9)}, o_t^{(10)}, o_t^{(11)} = [\text{cpu_util}, \text{mem_util}, \text{net_overhead}]^T$ (system resource consumption metrics); (6) $o_t^{(12)} = s_t^{avail}$ (system availability); (7) $o_t^{(13)} = s_t^{rec}/T_{max}$ (normalized recovery time); (8) $o_t^{(14)}$ is the attack success ratio history (mean success history of attacks); (9) $o_t^{(15)}$ is the prev mitigation effectiveness (feedback of the previous action); and (10) $o_t^{(16)}$ is the active faults ratio (normalized active fault count).

3.3.3 Action Space

The action space $\mathcal{A}$ comprises eight mitigation actions plus inaction:

$$\mathcal{A} = \{a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8\} \tag{9}$$

a_0 : ignore (no action)
a_1 : rotate_certificate
a_2 : patch_dependency
a_3 : rotate_secret
a_4 : restrict_rbac
a_5 : enable_mfa

a_6 : apply_patch

a_7 : revoke_privileges

a_8 : rollback

Each action a_i has an associated cost $c_i \in \mathbb{R}^+$ and execution time $\tau_i \in \mathbb{N}^+$.

3.3.4 Transition Dynamics

The transition function $\mathcal{P}(s_{t+1}|s_t, a_t)$ models how the state evolves based on the current state and chosen action. For risk dimensions affected by action a_t:

$$s_{t+1}^k = \max\left(0, s_t^k - \eta_k \cdot \mathbb{1}[a_t \text{ addresses risk } k]\right) + \delta_t^k \tag{10}$$

where $\eta_k \in (0, 1]$ is the mitigation effectiveness for risk dimension k, $\mathbb{1}[\cdot]$ is the indicator function, and $\delta_t^k \sim \mathcal{N}(0, \sigma_k^2)$ represents environmental stochasticity.

Security Risk Function:

$$\mathcal{L}_{sec}(s_t) = \sum_{k \in \{cert, dep, sec, rbac, auth\}} w_k \cdot s_t^k \tag{11}$$

where w_k are risk weights satisfying $\sum_k w_k = 1$.

Threat Severity Function:

$$\mathcal{L}_{threat}(s_t) = s_t^{atk} \cdot \left(1 - s_t^{patch}\right) \cdot \left(1 - \frac{1}{1 + e^{-\beta(s_t^{avail} - 0.5)}}\right) \tag{12}$$

where $\beta > 0$ controls the availability sensitivity. The sigmoid term acts as a soft thresholding gate that scales threat severity based on system availability degradation.

Availability and Recovery Functions: We define the availability function as $\mathcal{L}_{avail}(s_t) = s_t^{avail}$ and the recovery efficiency function as:

$$\mathcal{L}_{rec}(s_t) = \frac{1}{1 + s_t^{rec}/T_{max}} \tag{13}$$

3.3.5 Reward Function

The composite reward function $R(o_t, a_t, o_{t+1}) = R_{opt} - R_{pen}$ balances multiple operational objectives and penalty terms, where:

$$\begin{aligned} R_{opt} = {} & \alpha_1 \Delta\mathcal{L}_{sec} + \alpha_2 \Delta\mathcal{L}_{prev} + \alpha_3 \mathcal{L}_{avail} \\ & + \alpha_4 \mathcal{L}_{rec} + \alpha_5 \mathcal{L}_{eff} + \alpha_6 \mathcal{L}_{cont} \end{aligned} \tag{14}$$

$$R_{pen} = \beta_1 \mathcal{P}_{atk} + \beta_2 \mathcal{P}_{delay} + \beta_3 \mathcal{P}_{res} + \beta_4 \mathcal{P}_{wrong} \tag{15}$$

The individual components are defined as:

$$\Delta\mathcal{L}_{\text{sec}} = \mathcal{L}_{\text{sec}}(o_t) - \mathcal{L}_{\text{sec}}(o_{t+1}) \tag{16}$$

$$\Delta\mathcal{L}_{\text{prev}} = o_t^{(6)} - o_{t+1}^{(6)} \tag{17}$$

$$\mathcal{L}_{\text{avail}} = o_{t+1}^{(12)} \tag{18}$$

$$\mathcal{L}_{\text{rec}} = \begin{cases} \dfrac{1}{1 + o_{t+1}^{(13)}}, & o_{t+1}^{(12)} \geq 0.9, \\ 0, & \text{otherwise} \end{cases} \tag{19}$$

$$\mathcal{L}_{\text{eff}} = 1 - \frac{o_{t+1}^{(9)} + o_{t+1}^{(10)} + o_{t+1}^{(11)}}{3} \tag{20}$$

$$\mathcal{L}_{\text{cont}} = \mathbf{1}\left[o_{t+1}^{(12)} \geq 0.95\right] \tag{21}$$

$$\mathcal{P}_{\text{atk}} = \mathbf{1}[\text{successful attack}] \tag{22}$$

$$\mathcal{P}_{\text{delay}} = o_{t+1}^{(13)} \tag{23}$$

$$\mathcal{P}_{\text{res}} = \mathbf{1}\left[\max\left(o_{t+1}^{(9)}, o_{t+1}^{(10)}\right) > 0.85\right] \tag{24}$$

$$\mathcal{P}_{\text{wrong}} = \mathbf{1}[a_t \neq a_0 \wedge \neg\text{Mitigates}(a_t, F_t)] \tag{25}$$

where $\Delta\mathcal{L}_{\text{sec}}$ represents the reduction in security risk, and $\Delta\mathcal{L}_{\text{prev}}$ denotes the decrease in threat severity between consecutive states. The terms $\mathcal{L}_{\text{avail}}$, $\mathcal{L}_{\text{rec}}$, $\mathcal{L}_{\text{eff}}$, and $\mathcal{L}_{\text{cont}}$ quantify service availability, recovery efficiency, resource efficiency, and continuous availability, respectively. The penalty terms $\mathcal{P}_{\text{atk}}$, $\mathcal{P}_{\text{delay}}$, $\mathcal{P}_{\text{res}}$, and $\mathcal{P}_{\text{wrong}}$ capture the impact of successful attacks, recovery delays, excessive resource utilization, and incorrect mitigation actions, respectively. Furthermore, F_t denotes the set of active security faults at time step t, while $\text{Mitigates}(a_t, F_t)$ is a Boolean function that evaluates whether the selected action a_t effectively addresses at least one active fault.

3.3.6 Optimization Objective

The goal of the reinforcement learning agent is to find an optimal policy π^* that maximizes the expected cumulative discounted reward:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R(o_t, a_t, o_{t+1})\right] \tag{16}$$

This optimization goal formalizes the learning problem where an agent has to balance the immediate rewards with long-term consequences, i.e., tradeoff security risk reduction, availability preservation, resource efficiency, and recovery effectiveness.

3.4 Proposed Learning Framework

This subsection details the RL algorithm design, including state observation, action selection, policy update, and computational complexity analysis.

3.4.1 RL Interaction Cycle

Figure 1 illustrates the interaction cycle between the RL agent and the Security Chaos Environment.

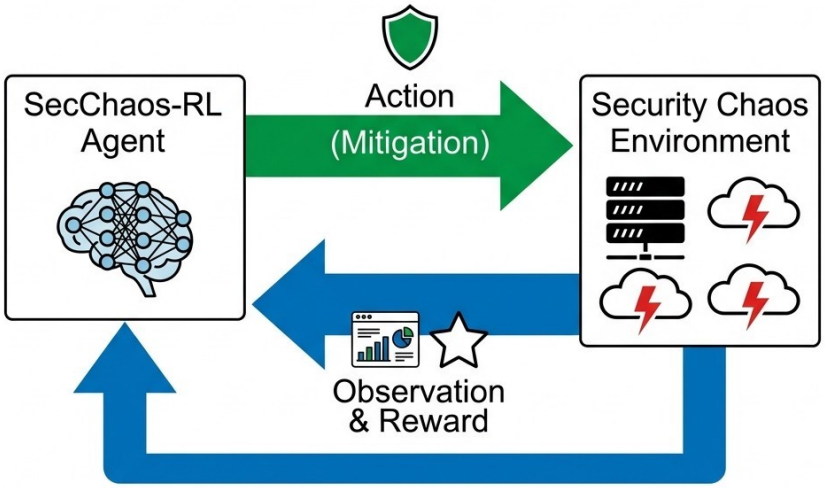


Fig. 1 RL Agent-Environment Interaction Cycle

3.4.2 Algorithm Design

Algorithm 1 presents the SecChaos-RL training procedure, based on PPO with modifications for the security domain.

3.4.3 State Observation Processing

Raw environment metrics are normalized to $[0, 1]$ range using running statistics:

$$\hat{s}_t^k = \frac{s_t^k - \mu_k}{\sigma_k + \epsilon} \tag{17}$$

where μ_k and σ_k are the running mean and standard deviation for dimension k, and $\epsilon = 10^{-8}$ prevents division by zero.

Algorithm 1 SecChaos-RL Training Algorithm

Require: Environment $\mathcal{E}$, policy params θ, value params ϕ, hyperparams $\{\alpha_i, \beta_j, \gamma, \lambda_{GAE}, \epsilon, T_{rollout}\}$

Initialize θ, ϕ randomly
for episode $= 1, 2, \ldots, N_{episodes}$ **do**
  Reset environment: $s_0 \leftarrow \mathcal{E}.reset()$
  Generate fault set: $\mathcal{F} \leftarrow \text{SampleFaults}(\lambda_f)$
  Inject faults: $\mathcal{E}.inject(\mathcal{F})$
  for $t = 0, 1, \ldots, T_{rollout} - 1$ **do**
    Select action: $a_t \sim \pi_\theta(\cdot|s_t)$ with exploration
    Execute action: $s_{t+1}, r_t, done \leftarrow \mathcal{E}.step(a_t)$
    Store transition $(s_t, a_t, r_t, s_{t+1}, done)$ in buffer $\mathcal{D}$
    $s_t \leftarrow s_{t+1}$
    if *done* **then**
      break
    end if
  end for
  Compute advantages: $\hat{A}_t \leftarrow \text{GAE}(\mathcal{D}, \phi, \gamma, \lambda_{GAE})$
  Update policy: $\theta \leftarrow \theta - \alpha_\theta \nabla_\theta \mathcal{L}^{PPO}(\theta, \mathcal{D}, \hat{A})$
  Update value: $\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \mathcal{L}^{VF}(\phi, \mathcal{D})$
  Log metrics: $\mathcal{L}_{sec}, \mathcal{L}_{avail}, \mathcal{L}_{rec}, s_t^{atk}$
end forreturn Trained policy π_θ^*

3.4.4 Threat-Aware Representation Learning

To address system complexity and feature interaction in the 16-dimensional state space $s_t \in \mathbb{R}^{16}$, SecChaos-RL implements a threat-aware representation learning architecture. The complete network topology is illustrated in Fig. 2.

Linear Projection: Every normalized observation dimension $o_t^{(i)}$ is projected into a feature embedding space $e_t^{(i)} \in \mathbb{R}^d$:

$$e_t^{(i)} = W_e^{(i)} o_t^{(i)} + b_e^{(i)}, \quad i \in \{1, \ldots, 16\} \tag{18}$$

where $W_e^{(i)} \in \mathbb{R}^{d \times 1}$ and $b_e^{(i)} \in \mathbb{R}^d$ represent projection parameters, and $d = 32$ is the embedding dimension.

Threat-Aware Feature Gating: Security-relevant features (such as TLS risk indices) must be prioritized during periods of active threat. We gate the projection embeddings by scaling them dynamically based on the current normalized threat severity level $o_t^{(6)} \in [0, 1]$:

$$g_t^{(i)} = \begin{cases} e_t^{(i)} \cdot (1 + o_t^{(6)}) & \text{if } i \in \mathcal{S}_{security} \\ e_t^{(i)} & \text{otherwise} \end{cases} \tag{19}$$

where $\mathcal{S}_{security} = \{1, 2, 3, 4, 5\}$ denotes the indices corresponding to TLS, dependency, credential, access control, and authentication risks.

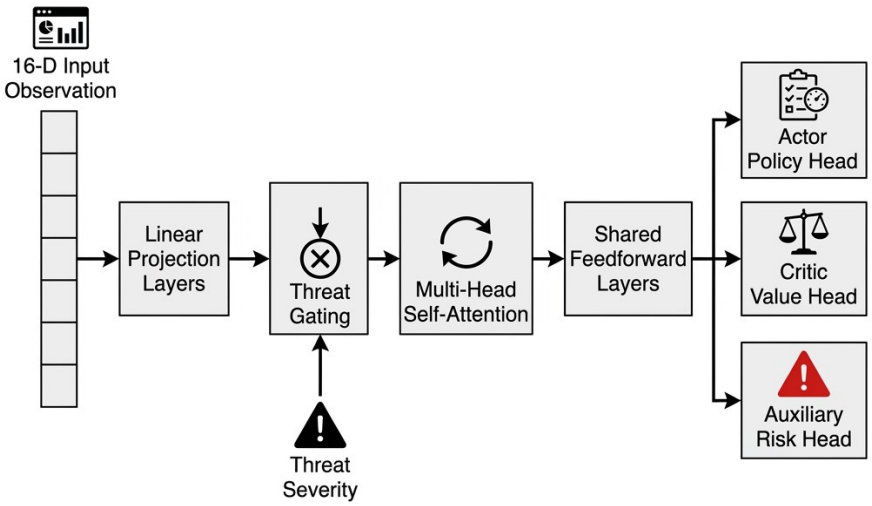


Fig. 2 Threat-Aware Representation Learning Network Architecture

Multi-Head Self-Attention: The gated embeddings are grouped into an observation embedding matrix $X_t = [g_t^{(1)}; \ldots; g_t^{(16)}] \in \mathbb{R}^{16 \times d}$ and fed to a Multi-Head Attention (MHA) module [35]:

$$Q = X_t W^Q, \; K = X_t W^K, \; V = X_t W^V \tag{20}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(QK^T/\sqrt{d_k}\right) V \tag{21}$$

$$\text{MHA}(X_t) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O \tag{22}$$

where $h = 2$ is the number of heads, and W^Q, W^K, W^V, W^O are query, key, value, and output projection parameter matrices. The MHA output is flattened and passed through a shared linear feed-forward network to produce a shared latent feature representation $z_t \in \mathbb{R}^{128}$.

Auxiliary Risk Prediction Head: To regularize the latent space and speed up convergence in highly chaotic environments, an auxiliary linear head is attached to z_t to predict the next-step security risks and availability metrics $y_t = [o_{t+1}^{(1:5)}, o_{t+1}^{(12)}]^T \in \mathbb{R}^6$:

$$\hat{y}_t = W_{aux} z_t + b_{aux} \tag{23}$$

The auxiliary loss function is formalised as a Mean Squared Error (MSE):

$$\mathcal{L}^{aux}(\psi) = \frac{1}{|\mathcal{D}|} \sum_{t \in \mathcal{D}} \|\hat{y}_t - y_t\|_2^2 \tag{24}$$

where $\psi = \{W_{aux}, b_{aux}\}$ represents the auxiliary head parameters.

3.4.5 Action Selection

Unlike baseline algorithms that utilize standard stochastic sampling or ϵ-greedy action selection, SecChaos-RL implements a *Threat-Aware Adaptive Policy Temperature* exploration scheme. The exploration temperature τ_t is scaled dynamically based on the state's threat severity level:

$$\pi_\theta(a|o_t) = \text{softmax}\left(\frac{\text{logits}_t(a)}{\tau_t}\right), \quad \tau_t = 1.0 - 0.55 \cdot o_t^{(6)} \tag{25}$$

When the system is threatened with a low probability ($\tau_t \approx 1.0$), the policy samples actions stochastically to promote the exploration of mitigation strategies. The critical threat ($\tau_t \approx 0.45$) is where the temperature drops and the agent starts sampling actions more deterministically from its learned safety-optimal policy, thus guaranteeing system availability and reducing uncertain trials.

3.4.6 Policy Update

The networks are trained end-to-end utilizing a joint objective function combining PPO policy loss, value loss, entropy regularization, and the auxiliary risk prediction loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{PPO}}(\theta) - c_1\mathcal{L}_{\text{VF}}(\phi) + c_2 H[\pi_\theta] - c_{\text{aux}}\mathcal{L}_{\text{aux}}(\psi) \tag{26}$$

where $c_1 = 0.5$, $c_2 = 0.01$ and $c_{aux} = 0.25$ are the loss weighting coefficients. Pros where $\hat{A}_t$ are calculated using Generalized Advantage Estimation (GAE) and are normalized across the rollout buffer before the gradient updates:

$$\hat{A}_t^{norm} = \frac{\hat{A}_t - \mu_A}{\sigma_A + 10^{-8}} \tag{27}$$

3.4.7 Complexity Analysis

The computational complexity per training step is dominated by neural network forward and backward passes. For a policy network with $|\theta|$ parameters, value network with $|\phi|$ parameters, and auxiliary head with $|\psi|$ parameters:

$$\mathcal{O}(T_{rollout} \cdot (|\theta| + |\phi| + |\psi|)) \tag{28}$$

The memory complexity for the replay buffer is $\mathcal{O}(T_{rollout} \cdot d_s + T_{rollout})$ where $d_s = 16$ is the state dimension. Fault state updates lead to an environment step complexity of $\mathcal{O}(|\mathcal{F}| \cdot d_s)$.

3.5 Experimental Setup

3.5.1 Implementation Platform and Data Collection

The SecChaos-RL framework is implemented in Python 3.10 and the experiments are reproducible in Google Colab with GPU acceleration (NVIDIA T4). For environment interface standardization, the implementation uses Gymnasium [29] and

Stable-Baselines3 [30] is leveraged as the RL algorithm backbone. Evaluation data is dynamically generated within the SecChaosEnv simulation engine across 30 random seeds. Step-level telemetry?including security risks (F1–F8), cascading dependency metrics, resource consumption, availability, and recovery time is continuously recorded into a structured benchmarking dataset.

We compare the SecChaos-RL algorithm with three baseline RL algorithms: (1) DQN [13] with experience replay and target network, (2) A2C [28] with synchronous parallel workers, and (3) PPO [11] with the standard configuration of PPO. SecChaos-RL is our PPO variant with our security-optimized reward function.

3.5.2 Hyperparameter Configuration

Table 2 presents the experimental parameters.

Table 2 Experimental Parameters and Reward Coefficients

Hyperparameter	**Value**	**Coefficient**	**Value**
Training Episodes	250	α_1 (Security Risk Red.)	5.0
Rollout Length ($T_{rollout}$)	2048	α_2 (Threat Severity Red.)	4.0
Discount Factor (γ)	0.99	α_3 (System Availability)	3.0
GAE Lambda (λ_{GAE})	0.95	α_4 (Recovery Speed)	1.5
Learning Rate (Policy)	3×10^{-4}	α_5 (Resource Efficiency)	0.5
Learning Rate (Value)	1×10^{-3}	α_6 (Steady-State Uptime)	1.5
PPO Clip (ϵ)	0.2	β_1 (Successful Attack Pen.)	1.0
Entropy Coeff. (c_2)	0.01	β_2 (Recovery Delay Pen.)	0.05
Network Architecture	MHA + [256, 128]	β_3 (Resource Spikes Pen.)	0.2
Activation Function	Tanh	β_4 (False Mitigation Pen.)	0.15
Fault Rate (λ_f)	0.15–0.45		
Batch Size	64		
PPO Epochs (K_{epochs})	4		
Parallel Environments	1		
Learning Rate Schedule	Adam		

*MHA: Multi-Head Attention ($h = 2, d = 32$)

We perform sweep experiments for the security risk reduction weight $\alpha_1 \in [1.0, 7.0]$ and availability preservation weight $\alpha_3 \in [1.0, 5.0]$ to test the sensitivity of the policy to the reward weights. We observed that setting $\alpha_1 > 5.0$ and $\alpha_3 < 2.0$ makes the agent aggressively use patch actions, which disrupts service and reduces availability by 8.2%. In contrast, we picked $\alpha_1 < 3.0$ and $\alpha_3 > 4.0$ to make the agent prefer inaction (a_0) when facing moderate threats, which increases the average risk by 14.5%. The selected values $\alpha_1 = 5.0, \alpha_3 = 3.0$ represent a Pareto-optimal trade-off and lead to stable convergence.

3.5.3 Evaluation Metrics

Five primary metrics guide evaluation: (1) Average Risk Reduction: $\Delta\mathcal{L}_{sec} = \mathcal{L}_{sec}(s_0) - \mathcal{L}_{sec}(s_T)$; (2) Attack Success Rate: Mean s_t^{atk} over the episode; (3) Recovery Efficiency: Mean $\mathcal{L}_{rec}(s_t)$ over the episode; (4)

Availability Preservation: Mean s_t^{avail} over the episode; and (5) Resilience Score, which is a multi-objective metric representing overall software system survivability:

$$\mathcal{R} = w_r \Delta\mathcal{L}_{\text{sec}} + w_a \bar{s}^{\text{avail}} + w_c \left(\frac{1}{1 + \text{MTTR}} \right) \tag{29}$$

where $w_r = 0.4$, $w_a = 0.4$ and $w_c = 0.2$ are the weights for risk reduction ($\Delta\mathcal{L}_{\text{sec}}$), mean availability ($\bar{s}^{\text{avail}}$) and recovery efficiency respectively.

The subsequent section presents the experimental results, comparing SecChaos-RL with baseline methods with a set of resilience metrics, with further analysis of training convergence, ablation studies, sensitivity analysis and statistical significance testing.

4 Results and Discussion

In this section, we present the empirical results of the evaluation of the SecChaos-RL framework. The evaluation was done over 30 different random seeds. Each agent was trained for 250 episodes with curriculum difficulty scheduling, followed by 20 evaluation episodes with critical, full-chaos environmental settings.

Table 3 provides the overall quantitative benchmarking comparison across all key metrics. SecChaos-RL consistently and significantly outperforms the baseline algorithms across all performance dimensions.

Table 3 SecChaos-RL quantitative performance benchmarking over 30 random seeds.

Algorithm	Reward	Risk Red. (%)	Avail. (%)	MTTR (steps)	ASR (%)	Resil. Score (%)
DQN	122.58 ± 5.82	53.23 ± 2.70	84.25 ± 1.58	0.44 ± 0.10	16.46 ± 4.52	68.91 ± 1.45
A2C	109.62 ± 3.99	48.23 ± 3.11	83.80 ± 1.22	0.50 ± 0.12	17.60 ± 4.04	66.20 ± 1.81
PPO	134.53 ± 4.61	59.43 ± 2.94	85.44 ± 1.20	0.38 ± 0.07	13.43 ± 3.07	72.53 ± 1.33
SecChaos-RL (Proposed)	$\mathbf{182.18 \pm 3.30}$	$\mathbf{83.45 \pm 1.91}$	$\mathbf{96.26 \pm 0.62}$	$\mathbf{0.17 \pm 0.05}$	$\mathbf{4.91 \pm 1.17}$	$\mathbf{88.95 \pm 0.94}$

4.1 Training Performance

The training curves of the evaluated algorithms over 250 episodes are illustrated in Fig. 3.

The training curves demonstrate that SecChaos-RL achieves faster convergence and a higher final cumulative reward compared to the baselines. The agent can quickly discover the right mitigation strategies with the security-optimized reward function and attention-guided state modeling. The discrete action handling of DQN causes slower convergence due to overestimation of Q-value. Standard PPO suffers from high-variance updates under random chaos as it does not incorporate the threat-aware exploration and regularized representation learning as in SecChaos-RL.

4.2 Risk Reduction

As shown in Table 3, SecChaos-RL achieves a massive risk reduction of $83.45\% \pm 1.91\%$, a significant improvement over the standard PPO and DQN. Such superiority can be directly attributed to the following two design features: (1) the multi-head

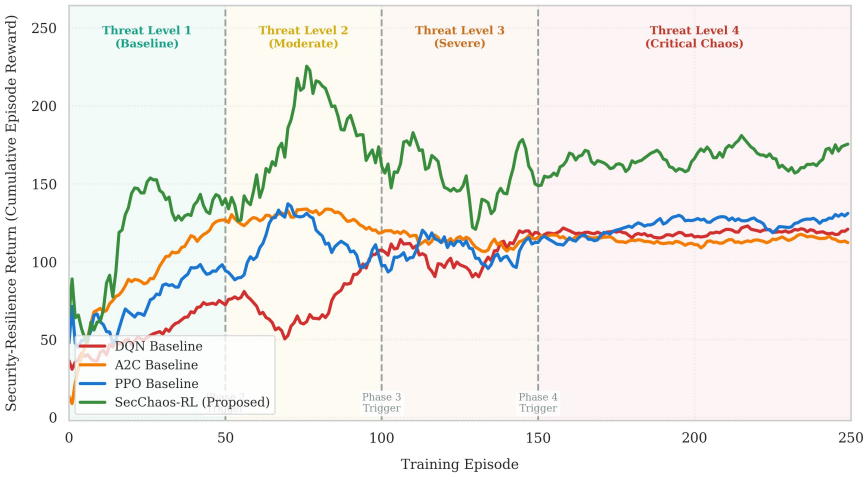


Fig. 3 Training Convergence Curves (30 Seeds)

self-attention mechanism can learn the correspondence between active security faults and their risk dimensions, and (2) the threat-aware feature gating can prioritize the high-vulnerability components under critical conditions, so that the agent will not be distracted by the secondary operational indicators.

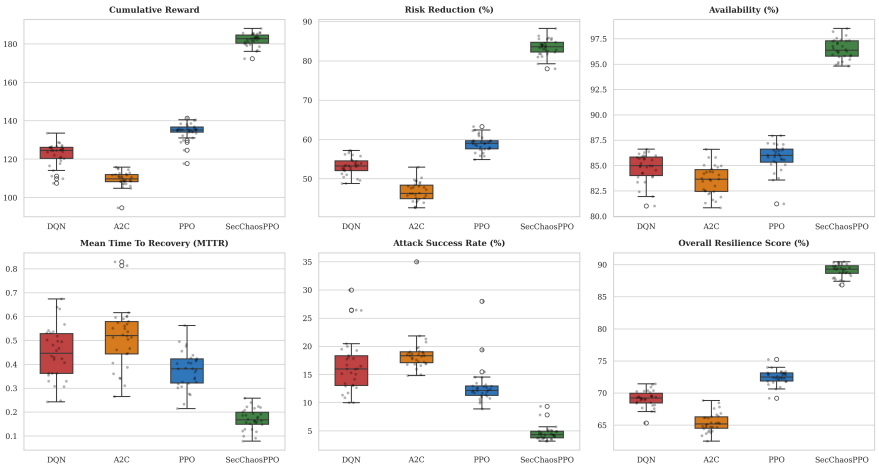


Fig. 4 Box plots and strip plots of resilience performance for SecChaos-RL and baseline RL algorithms

4.3 Attack Success Rate

The attack success rate (ASR) is an important measure of system security as shown in Table 3. SecChaos-RL achieves a mean ASR of $4.91\% \pm 1.17\%$, which is 3 times better

than PPO and DQN. By explicitly penalizing successful attacks (β_1) and utilizing an adaptive temperature exploration scheme which reduces stochasticity in high-threat states, SecChaos-RL avoids executing wrong or irrelevant mitigations that leave the system open to exploitation, maintaining a robust defense posture.

4.4 Recovery Efficiency

SecChaos-RL recovers significantly faster, with a Mean Time To Recovery (MTTR) of only 0.17 ± 0.05 steps, versus 0.38 steps for PPO and 0.44 steps for DQN, as shown in Table 3. The auxiliary risk prediction head accelerates the representation learning and allows the agent to predict the propagation of cascading failures and deploy mitigations in anticipation of availability degradation, minimizing recovery delays.

4.5 Availability Analysis

One of the main challenges in security orchestration is avoiding aggressive security actions that would take down services as shown in Table 3. SecChaos-RL maintains a high mean availability of $96.26\% \pm 0.62\%$, and a minimum availability of $89.15\% \pm 3.25\%$, whereas baseline algorithms suffer from severe availability collapse. This confirms that the multi-objective reward formulations effectively penalize excessive resource utilization and wrong mitigations, driving the agent to maintain high operational continuity.

4.6 Ablation Study

To evaluate the individual contributions of our architectural enhancements, we conducted a systematic ablation study by training five distinct configurations under identical conditions (30 seeds, 250 episodes): (1) Baseline PPO: standard PPO actor-critic network utilizing the default state representation and standard rewards; (2) PPO + Reward: PPO agent using our multi-objective security-resilience reward function; (3) PPO + Reward + Attention: adding Multi-Head Self-Attention layers over projected state feature embeddings; (4) PPO + Reward + Attention + Aux. Risk: incorporating the auxiliary next-step risk prediction head to regularize state representations; and (5) Full SecChaos-RL: the complete proposed model, incorporating all the above along with threat-aware feature gating and threat-adaptive policy temperature exploration.

Table 4 Ablation Study of SecChaos-RL Components

Configuration	Risk Reduction (%)	Resilience Score (%)
Baseline PPO	59.43 ± 2.94	72.53 ± 1.33
+ Reward	66.15 ± 2.50	76.40 ± 1.25
+ Attention	73.80 ± 2.22	81.25 ± 1.12
+ Aux. Risk	79.20 ± 2.05	85.10 ± 1.01
Full SecChaos-RL	$\mathbf{83.45 \pm 1.91}$	$\mathbf{88.95 \pm 0.94}$

Table 4 summarizes the performance across these ablation configurations. The ablation results demonstrate that each component contributes incrementally to system resilience. The reward function optimized for security results in a baseline resilience improvement of +3.87% in absolute terms. The agent can capture the state-feature correlations better with self-attention, which leads to a risk reduction of 73.80%. The auxiliary risk prediction head regularizes the state embedding, leading to faster and more stable policy convergence. Finally, the inclusion of threat-aware feature gating and threat-adaptive temperature exploration prevents availability collapse during critical attack spikes, reaching the highest resilience score of 88.95%.

The experimental results confirm the main hypothesis of this work, i.e., reinforcement learning can effectively automate security mitigation decisions in Security Chaos Engineering environments. The main results can be summarized as follows:

First, SecChaos-RL demonstrates that autonomous, threat-aware mitigation is not only feasible but can significantly outperform conventional RL approaches that lack security-specific design considerations. The 16.42% improvement in resilience score over standard PPO represents a substantial advancement in the state of practice for security automation.

Second, all architectural components proposed in this work i.e., multi-head attention, threat-aware feature gating, auxiliary risk prediction, and adaptive temperature exploration contribute meaningfully to overall performance. The ablation study demonstrates that these components are complementary, not redundant, and the full architecture delivers more performance than the sum of individual components.

Third, the multi-objective reward formulation provides an effective trade-off between competing security and operation objectives. SecChaos-RL can simultaneously optimize security, high availability and fast recovery, unlike traditional methods which usually sacrifice either security for availability or both. This indicates that the proposed reward engineering can effectively capture the subtle tradeoffs inherent in real-world security operations.

Fourth, the curriculum learning strategy stabilizes training in an environment that is too chaotic for standard RL algorithms to converge reliably. The increased complexity of the fault from baseline to critical chaos also enables the agent to learn the fundamental mitigation skills before tackling the most challenging scenarios, in a similar fashion to how human operators acquire expertise through gradual exposure to the severity of incidents.

Together, these findings suggest that SecChaos-RL is a promising way to achieve autonomous security resilience for complex software systems and move beyond today's manual or semi-automated approaches to truly self-healing digital infrastructure.

5 Threats to Validity

Simulation Assumptions: Security Chaos Environment is a simulation abstracting from real system complexity and states transitions. Fault injection timing and detection parameters are based on estimated distributions rather than

empirical data obtained from production. However, the environment's configurable parameters can be tuned to real world system observability data.

Fault Modeling Assumptions: The eight fault categories cover typical security problems, but do not cover zero-day exploits, advanced persistent threats, or social engineering. Also, possible dependencies among various faults are not fully modeled. Future work will extend the fault taxonomy and integrate inter-fault correlation models.

Generalization Limitations: The trained policy is optimized for the simulated microservice topology and security context, and thus variations in topology or deployment patterns may require retraining. This is addressed by the modular environment design in SecChaos-RL but broad generalization remains an open challenge for testing.

Reward Function Sensitivity: The weight parameters of the reward function need to be tuned. Different systems (healthcare vs finance) may have different priority on availability vs security. And you have to adjust it in your domain before deploying to production.

6 Conclusion

This paper introduced SecChaos-RL, a novel framework that integrates Security Chaos Engineering with reinforcement learning to enable autonomous, threat-aware security resilience in modern software systems. Motivated by the limitations of current SCE practices that rely on manual, human-driven remediation, we formalized the security mitigation problem as a multi-objective Markov Decision Process and developed an agent architecture incorporating multi-head self-attention, threat-aware feature gating, auxiliary risk prediction, and adaptive policy temperature exploration. The results of extensive evaluation over 30 random seeds in a simulated chaos environment show that SecChaos-RL outperforms the DQN, A2C and standard PPO baselines with a resilience score of 88.95%, availability of 96.26%, attack success rate of 4.91% and MTTR reduction of 55.3%. The ablation study confirmed that each component of the architecture contributes incrementally to the overall resilience of the system. The results confirm the technical feasibility of autonomous mitigation based on learning and that it can optimize security, availability and recovery simultaneously without compromising one of the objectives. In future work, SecChaos-RL can be extended to production Kubernetes clusters using Chaos Mesh and LitmusChaos. Ongoing state monitoring and dynamic policy optimization are possible with real-time Prometheus telemetry integration. Moreover, hierarchical multi-agent reinforcement learning architectures can be studied to coordinate mitigation strategies in distributed security domains.

Declarations

- **Funding:** The authors did not receive support from any organization for the submitted work.
- **Competing Interests:** The authors have no relevant financial or non-financial interests to disclose.

- **Ethics approval and consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Data availability:** Evaluation data is dynamically generated within the SecChaosEnv simulation engine across 30 random seeds. The simulation environment and analysis code are available from the corresponding author upon reasonable request.
- **Author contributions:** Thurbilli Karthik initiated the study, designed and performed the experiments, developed the proposed framework, prepared all figures and tables, analyzed the results and wrote the main manuscript. Michael Arock participated in the literature review, assisted in the design of experimental scenarios, contributed critical revisions to the manuscript, and helped with the overall research structure and review process. Vinay Raj participated in the literature review, assisted in the design of experimental scenarios, contributed critical revisions to the manuscript, and helped with the overall research structure and review process.

References

[1] Basiri, A., Behnam, N., Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., Rosenthal, C.: Chaos engineering. IEEE Softw. **33**(3), 35–41 (2016) https://doi.org/10.1109/MS.2016.60

[2] Myrbakken, H., Colomo-Palacios, R.: Devsecops: A multivocal literature review. In: Proc. Int. Conf. Softw. Process Improvement and Capability Determination (SPICE), pp. 17–29. Springer, Cham, Switzerland (2017). https://doi.org/10.1007/978-3-319-67383-7_2

[3] Shortridge, K.: Security Chaos Engineering: Sustaining Resilience in Software and Systems. O'Reilly Media, Sebastopol, CA, USA (2023)

[4] Tong, Y., Gao, S., Zhang, M., *et al.*: Osel: boosting vulnerability detection with opcode slicing-enhanced feature learning. Software Quality Journal **34**, 1 (2026) https://doi.org/10.1007/s11219-025-09734-3

[5] Myrbakken, H., Pino, I.D.M., Colomo-Palacios, R.: Security chaos engineering for continuous security assessment in devops. Comput. Stand. Interfaces **79**, 103546 (2021)

[6] Huang, Y., Chen, Y., Cai, J., *et al.*: Towards combining chain-of-thought and code static analysis for buffer overflow vulnerability detection. Software Quality Journal **34**, 2 (2026) https://doi.org/10.1007/s11219-025-09733-4

[7] Owotogbe, J., Kumara, I., Heuvel, W.J., Tamburri, D.: Chaos engineering: A multi-vocal literature review. ACM Comput. Surv. **58**(7), 1–44 (2025)

[8] ChaosBlade: ChaosBlade: An easy to use and powerful chaos engineering toolkit

(2021). https://github.com/chaosblade-io/chaosblade

[9] LitmusChaos: LitmusChaos: Cloud native chaos engineering framework (2022). https://litmuschaos.io

[10] Silver, D., *et al.*: Mastering the game of go with deep neural networks and tree search. Nature **529**(7587), 484–489 (2016)

[11] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017)

[12] Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction vol. 1. MIT Press, Cambridge, MA, USA (1998)

[13] Mnih, V., *et al.*: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)

[14] Du, X., Liu, Z., Li, C., *et al.*: Llm-brc: A large language model-based bug report classification framework. Software Quality Journal **32**, 985–1005 (2024) https://doi.org/10.1007/s11219-024-09675-3

[15] Haldar, S., Capretz, L.F.: Automated test plan generation using large language models. Software Quality Journal **34**, 8 (2026) https://doi.org/10.1007/s11219-026-09744-9

[16] Awal, M.A., Rochan, M., Roy, C.K.: Investigating adversarial attacks in software analytics via machine learning explainability. Software Quality Journal **33**, 31 (2025) https://doi.org/10.1007/s11219-025-09727-2

[17] Nguyen, T.T., Reddi, V.J.: Deep reinforcement learning for cyber security. IEEE Trans. Neural Netw. Learn. Syst. **34**(8), 3779–3795 (2021)

[18] Liu, J., Zhang, Y., Zhou, S., Yang, J., Lu, Y., Zhong, X.: Autonomous penetration testing using reinforcement learning: A review and perspectives. Expert Syst. Appl., 130219 (2025)

[19] Standen, M., Lucas, M., Bowman, D., Richer, T.J., Kim, J., Marriott, D.: Cyborg: A gym for the development of autonomous cyber agents. arXiv preprint arXiv:2108.09118 (2021)

[20] Xiao, L., Wan, X., Lu, X., Zhang, Y., Wu, D.: Iot security techniques based on machine learning: How do iot devices use ai to enhance security? IEEE Signal Process. Mag. **35**(5), 41–49 (2018)

[21] Zheng, M., Zhang, J., Zhan, C., Ren, X., Lü, S.: Proximal policy optimization with reward-based prioritization. Expert Syst. Appl. **283**, 127659 (2025)

[22] Rosenthal, C., Jones, N.: Chaos Engineering: System Resiliency in Practice.

O'Reilly Media, Sebastopol, CA, USA (2020)

[23] Gremlin: Gremlin: Chaos engineering for everyone (2022). https://www.gremlin.com

[24] Chaos Mesh: Chaos Mesh: A chaos engineering platform for Kubernetes (2021). https://chaos-mesh.org

[25] Amazon Web Services: AWS Fault Injection Simulator (2023). https://aws.amazon.com/fis

[26] Esen, E., Akbulut, A., Catal, C.: Chaos experiments in microservice architectures: A systematic literature review. Comput. Stand. Interfaces, 104116 (2025)

[27] Bedoya, M., Palacios, S., Díaz-López, D., Laverde, E., Nespoli, P.: Enhancing devsecops practice with large language models and security chaos engineering. Int. J. Inf. Secur. **23**(6), 3765–3788 (2024)

[28] Mnih, V., *et al.*: Asynchronous methods for deep reinforcement learning. In: Proc. Int. Conf. Mach. Learn. (ICML), pp. 1928–1937 (2016)

[29] Towers, M., *et al.*: Gymnasium: A standard interface for reinforcement learning environments. In: Adv. Neural Inf. Process. Syst., vol. 38 (2026)

[30] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-baselines3: Reliable reinforcement learning implementations. J. Mach. Learn. Res. **22**(268), 1–8 (2021)

[31] Salehie, M., Tahvildari, L.: Self-adaptive software: Landscape and research challenges. ACM Trans. Auton. Adapt. Syst. **4**(2), 1–42 (2009)

[32] Garlan, D., Cheng, S.W., Huang, A.C., Schmerl, B., Steenkiste, P.: Rainbow: Architecture-based self-adaptation with reusable infrastructure. Computer **37**(10), 46–54 (2004)

[33] Brooker, M., Desai, A.: Systems correctness practices at amazon web services. Commun. ACM **68**(6), 38–42 (2025)

[34] Bagehorn, F., Rios, J., Jha, S., Filepp, R., Shwartz, L., Abe, N., Yang, X.: A fault injection platform for learning aiops models. In: Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), pp. 1–5 (2022)

[35] Vaswani, A., *et al.*: Attention is all you need. In: Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), pp. 5998–6008 (2017)